

OOP Programming Assignment

JUNE 2024

1. General Instructions

The general guideline for this assignment is as follows:

1. Evidence of **plagiarism or collusion** will be taken seriously, and University regulations will be applied fully to such cases, in addition to **ZERO marks** being awarded to all parties involved.
2. The total mark for the assignment is 105, and it contributes 20% to the total grade.
3. This is a **group assignment** with a maximum of 2 students per group. Students are free to choose their group members, and group members can be from different practical groups.
4. The deadline for assignment submission is **6 September 2024, Friday, before 5:00 pm** via WBLE.
5. Students must submit **ONE (1)** copy of the complete **JAVA source code files (.java)** with **UML class diagrams (.docx or .pdf)** and **ONE (1) presentation video (10 mins max)** briefly explaining each class's function and the output of the application. Please compress all the files into a zip file (*.zip) and ensure the **file size does not exceed 30MB**. Then, rename the folder as "Group_YourGroupNumber.zip". For example, if your registered group number via the shared Excel spreadsheet in WBLE is 1, then your zip folder should be named as:
Group1.zip
6. **Late submission** after the due time/date may incur a late penalty as shown below:
 - i. Late submission on the same day – 5 marks deduction
 - ii. Late submission of one day – 10 marks deduction
 - iii. Late submission of more than a day – WILL NOT BE ACCEPTED as a submission

2. Background

In this assignment, you are required to develop a Hospital Management System (HMS) in Java using Eclipse IDE and object-oriented programming concepts. The main objective of the HMS is to manage activities in a hospital, including patients, doctors, administrative staff, medicine, facilities, and laboratories. Therefore, this HMS should contain tables for all records of patients, doctors, staff (i.e., nurses, pharmacists, security, etc.), medicine, facilities, and laboratories.

3. Requirements

Your assignment is to develop a program to manage the HMS described below.

3.1 Program Functions (25 marks)

Your program must fulfill the following functional specifications:

- Display the Welcome to the HMS and the current date and time.

- Display the main menu contains all the six selections (Doctors, Patients, Medicine, Laboratories, Facilities, Staff).
- Declare, create, and initialize array/ArrayList named `doctors`, `patients`, `medicine`, `laboratories`, `facilities`, and `staff`. You are free to give any name during array initialization, but you are required to initialize at least **three values** for each of the array. The **first two values (id and names)** in the `doctors` array in the `doctors` array should be your **group members' names and IDs** (use the last three numbers of your student ID).
- Provide intuitive input method that allows users to add new entry or to display existing list from corresponding arrays of the classes.
- Check if the user would like to return to previous selection (i.e., to add another new entry), return to main menu (i.e., containing all six selections) or exit the program.

3.2 Object-Oriented Design (40 marks)

You must use Java and object-oriented programming techniques in your implementation. Your design should follow good object-oriented design principles such as:

- Single responsibility – a class should have responsibility for a single functionality of a program and that responsibility should be encapsulated by the class.
- Open/closed principle – classes or modules should be open for extension but closed for modification.
- Efficient and no redundancy – keep it simple.

The followings are the basic classes that **MUST** exist in your program. The responsibilities of each of the classes, as well as some of their properties and methods are suggested. You are free to select the data types and method signatures for the classes, and to add additional classes if necessary.

Staff:

Staff is the class for representing a staff object. The `Staff` class contains:

- Four `String` data fields named `id`, `name`, `designation`, and `sex`. A field named `salary` that stores `salary (int)` of the staff.
- A method named `newStaff()` that prompts user to enter staff's `id`, `name`, `designation`, `sex` and `salary`.
- A method name `showStaffInfo()` to show the content of the staff as:

<code>[id]</code>	<code>[name]</code>	<code>[designation]</code>	<code>[sex]</code>	<code>[salary]</code>
-------------------	---------------------	----------------------------	--------------------	-----------------------

Doctor:

Doctor is the class for representing a doctor object. The `Doctor` class contains:

- Five `String` data fields named `id`, `name`, `specialist`, `workTime`, and `qualification`. A field named `room` that stores `room number (int)` of the doctor.
- A method named `newDoctor()` that prompts user to enter doctor's `id`, `name`, `specialization`, `work time`, `qualification` and `room number`.

- A method name `showDoctorInfo()` to show the content of the doctor as:
[id] [name] [specialist] [work time] [qualification] [Room No.]

Patient:

Patient is the class for representing a patient object. The **Patient** class contains:

- Five String data fields named `id`, `name`, `disease`, `sex`, and `admitStatus`. A field named `age` that stores age (`int`) of the patient.
- A method named `newPatient()` that prompts user to enter patient's `id`, `name`, `disease`, `sex`, `admit status` and `age`.
- A method name `showPatientInfo()` to show the content of the patient as:
[id] [name] [disease] [sex] [admitStatus] [age]

Medicine:

Medicine is the class for representing a medicine object. The **Medicine** class contains:

- Three String data fields named `name`, `manufacturer`, and `expiryDate`. Two `int` data fields named `cost` and `count`.
- A method named `newMedicine()` that prompts user to enter `name`, `manufacturer`, `expiry date`, `cost` and `number of unit`.
- A method name `findMedicine()` to show the content of the medicine as:
[name] [manufacturer] [expiry date] [cost]

Lab:

Lab is the class for representing a lab object. The **Lab** class contains:

- A String data fields named `lab`. A data fields named `cost` that stores `cost (int)` of the facility.
- A method named `newLab()` that prompts user to enter `facility` and `cost`.
- A method name `labList()` to show the content of the lab as:
[lab] [cost]

Facility:

Facility is the class for representing a facility object. The **Facility** class contains:

- A String data fields named `facility`.
- A method named `newFacility()` that prompts user to enter `facility`.
- A method name `showFacility()` to show the content of the facility as:
[facility]

HospitalManagement:

The **HospitalManagement** class is the application class which controls the flow of the HMS, displays messages and gets inputs from the users. It should contain the **main()** method and statements to:

- Declare and create an array of **Doctor** objects. Number of doctors in the hospital is 25.
- Declare and create an array of **Patient** objects that can hold 100 values.
- Declare and create an array of **Lab** objects that can hold 20 values.
- Declare and create an array of **Facility** objects that can hold 20 values.
- Declare and create an array of **Medicine** objects that can hold 100 values.
- Declare and create an array of **Staff** objects that can hold 100 values.
- Provide existing lists for doctors, patients, labs, facilities, medicine, and staffs, with a minimum number of three objects for each class.
- Implement selection controls and/or loops to support the program functions described in Section 3.1.

3.3 Interface (5 marks)

- Application extension – Extend the application by adding new type of staff class(es). A good design should allow extending the application with minimum changes to the main components of the program.
- Add exception handling to handle runtime errors (for example, incorrect data type of the input value entered by user)
- Graphical user interface (GUI) – Enhance the application by providing easy to use and nice user interface with the use of **JavaFX** technology.

3.4 Recording Video (35 marks)

- Introductions: Your name, student ID, title of assignment and show face.
- Program description, including what is the program about and how it works as well as the test cases (program outputs).
- UML class diagrams showing the program structure.